

Ratchet: A Minimal Hygiene Recipe for Self-Evolving LLM Agents*

Xing Zhang¹ Yanwei Cui¹ Guanghui Wang¹ Ziyuan Li²
Wei Qiu² Bing Zhu² Peiyang He^{1†}

¹AWS Generative AI Innovation Center ²HSBC Holdings Plc., HSBC Technology Center, China

Abstract

Self-evolving skill libraries, pioneered by Voyager (1), let frozen LLM agents accumulate reusable knowledge without weight updates, yet recent evaluation shows that LLM-authored skills deliver +0.0pp over no-skill baselines while human-curated ones deliver +16.2pp (2): the bottleneck is not skill authoring but lifecycle management. We introduce **Ratchet**, a single-agent loop in which a frozen LLM writes, retrieves, curates, and retires its own natural-language skills. Ratchet integrates four candidate hygiene mechanisms: outcome-driven retirement, a bounded active-cap, meta-skill authoring guidance, and pattern canonicalisation. On MBPP+ hard-100 with Claude Opus 4.7, Ratchet lifts held-out pass@1 from a 0.258 ± 0.047 baseline to a late-window rolling mean of 0.584 (peak 0.658 ± 0.042) across 100 rounds and 3 seeds, a $+0.328 \pm 0.018$ rolling-mean gain where the no-skill control drifts at $+0.002 \pm 0.005$; the same recipe transfers to an agentic solver on SWE-bench Verified (+0.22 peak lift over 20 rounds). Eight ablations (A1–A8) reveal that the minimal working recipe is smaller than our design suggests: retirement and the meta-skill authoring prior are load-bearing, while explicit deduplication (canonicalisation, cover-guard) is subsumed by the meta-skill itself. A non-divergence proposition shows that bounded cap and retirement threshold together prevent expected performance from drifting below the no-skills floor.

1 Introduction

A frozen LLM that solves a hard task today learns nothing that helps it tomorrow. Bridging this gap without weight updates requires an external memory the same model can consult before its next attempt. Recent work (3) positions such memories on an *experience compression spectrum*, from raw traces (L_0 , e.g., Reflexion (4)) through episodic memories (L_1 , e.g., MemGPT (5)) and procedural skills (L_2 , e.g., Voyager (1)) to declarative rules (L_3 , e.g., RuleShaping (6)), and aggregates a pointed finding: *human-curated* skills deliver +16.2pp over a no-skill baseline, while *LLM-self-generated* skills deliver +0.0pp (SkillsBench (2)). The bottleneck, across two dozen surveyed systems, is not the author but the **librarian**: lifecycle management (versioning, conflict detection, deprecation) is “largely neglected.” We term this gap *library drift*: the silent degradation of a skill library’s effective quality through unchecked growth, redundancy, or premature pruning. Self-evolution is the right direction, consistent with the bitter lesson (7), but our Proposition 1 (Sec. 3.5) shows that without a bounded active-cap and a retirement threshold the worst-case performance bound collapses to vacuous. The question is not *whether* to self-evolve but what minimal structure makes self-evolution non-divergent.

*An earlier workshop version of this work is available at arXiv:2605.19576. This paper extends that version with the SWE-bench Verified evaluation and the non-divergence analysis (Proposition 1).

[†]Corresponding author: peiyan@amazon.com.

Ratchet is a single-agent loop in which a frozen LLM writes, retrieves, curates, and retires its own natural-language skills, making the librarian explicit via four governance mechanisms. *Pattern canonicalisation* collapses near-duplicate critic labels so that two failure descriptions of the same bug produce one skill, not two. *Contribution-score retirement* with an evidence floor demotes marginal skills on measured outcomes, replacing the heuristic update rules of prior systems (1; 8; 9). A *bounded active-cap* forces competition for the Router’s shortlist so retrieval does not degrade as the bank grows. A *meta-skill authoring prior* constrains the Synthesizer to produce stylistically consistent skills, implicitly deduplicating without a separate filter. Because skills are synthesised from failure clusters rather than successes, they are naturally pitfall-oriented, inheriting the guardrails-beat-guidance effect found for L_3 rules (6; 10).

On MBPP+ hard-100 (11) with Claude Opus 4.7, Ratchet lifts 10-round rolling-mean held-out pass@1 from 0.258 ± 0.047 to a late-window 0.584 over 100 rounds (3-seed mean gain $+0.328 \pm 0.018$), while a forced-no-skill control posts $+0.002 \pm 0.005$.

Contributions

- **Thesis:** we reframe SkillsBench’s (2) null result. The bottleneck in self-evolving skill libraries is not authoring quality but *lifecycle management*: retirement, capacity control, and authoring guidance.
- **Empirical:** addressing the librarian bottleneck alone (frozen author, no weight update) yields a $+0.328$ rolling-mean gain on MBPP+ hard-100 and $+0.22$ peak lift on SWE-bench Verified with an agentic solver, to our knowledge the first LLM-self-authored library that closes the gap to human-curated performance.
- **Mechanistic:** eight ablations identify what “librarian” means concretely. Outcome-driven retirement and the meta-skill authoring prior are load-bearing (-0.141 when the prior is removed; -0.019 under harsh retirement); explicit deduplication is subsumed by the authoring prior at this scale; frequent meta-skill refresh (A8) trades 55% wall-time for marginal gain.
- **Theoretical:** a non-divergence bound (Prop. 1) formalises why cap and threshold are the minimum governance: expected pass@1 cannot drift below the no-skill floor by more than $\tau + \epsilon + C\delta$, finite only because both are finite.

2 Related Work

Episodic self-correction. Reflexion (4), Self-Refine (12), and ReAct (13) reflect or critique within a single episode and discard state at the boundary. Ratchet lifts this pattern to cross-task persistence.

Cross-task skill libraries. Voyager (1) pioneered the ever-growing skill library with executable Minecraft skills; ExpeL (8) extracts textual insights from trajectories but lacks outcome-driven retirement; AutoManual (9) compiles a flat instruction manual per domain. DSPy (14), OPRO (15), and TextGrad (16) optimise prompts but do not persist per-skill evidence. MemGPT (5) and Generative Agents (17) manage memory hierarchies but define no typed, retirable skill artifact. Concurrent 2025–2026 work advances the skill-library frontier along orthogonal axes: EvolveR (18) distills trajectories into strategic principles; Trace2Skill (19) induces skills from trace pools via parallel sub-agents; CASCADE (20) pairs meta-skills with cumulative skill creation; AutoSkill (21) provides the closest lifecycle machinery (versioning + standardised schema) but does not retire on measured per-task contribution; SkillRL (22) couples skill libraries with RL weight updates (a different regime from frozen-LLM systems); SSL (23) decomposes skills into scheduling, structural, and logical layers; and Strategy Genes (24) attaches failure history as first-class metadata. A recent survey (3) of 20+ such systems reports that lifecycle management is largely absent; none adopts outcome-driven retirement paired with a bounded active-cap. Table 1 disaggregates the comparison into eight design features spanning signal quality, authoring, and governance. Our A1–A8 ablations (Sec. 4.3) isolate what happens when Ratchet itself is stripped of each.

Design-axis parallels between ablations and prior systems. Several of our ablations (Sec. 4.3) approximate a prior system along one axis: A7 ($C = 100$) moves toward Voyager’s unbounded

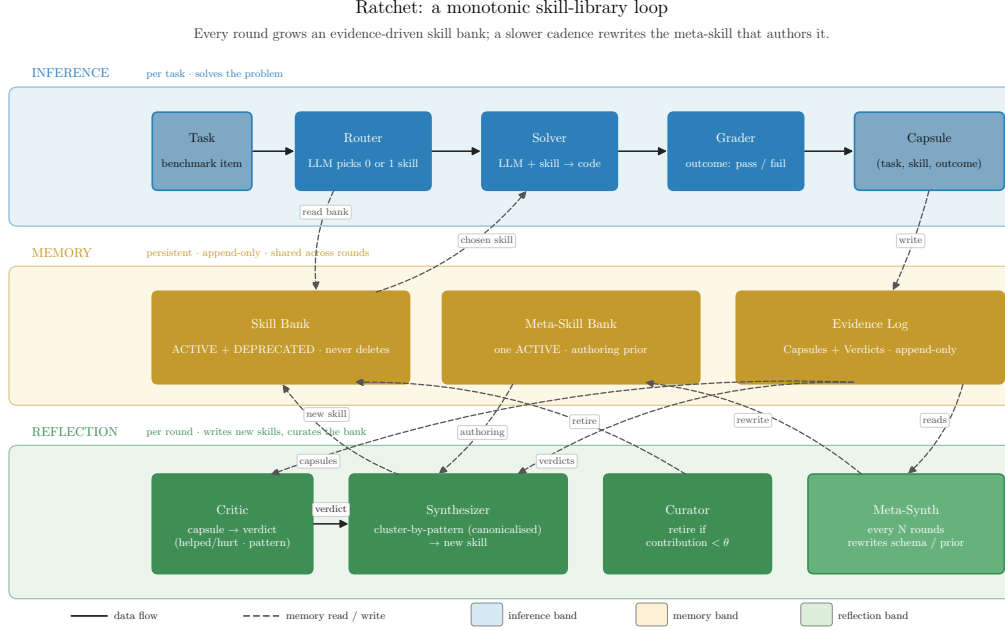


Figure 1: The Ratchet loop. **Inference** (top): each task flows through Router → Solver → Grader → Capsule. **Memory** (middle): three append-only stores (Skill Bank, Meta-Skill, Evidence Log). **Reflection** (bottom): every round the Critic labels failures, the Synthesizer writes new skills from failure clusters, and the Curator retires under-performers. Solid arrows = data flow; dashed = memory reads/writes.

library; A3 (no-meta) removes the authoring-prior that distinguishes Ratchet from AutoManual; A5 (no canonicalisation) reverts toward ExpeL’s free-form labels. These parallels motivate the ablations but are not claims that any ablation *is* the prior system; each differs from Ratchet on multiple axes simultaneously.

Critic reliability and lifecycle. Zheng et al. (26) report >80% agreement between strong LLM judges and human preferences; we further constrain the Critic to a closed label set and require three failures sharing a canonical pattern before any skill is born. Weight-update continual learning uses regularisers like EWC (27); Ratchet avoids weight-update forgetting entirely (weights frozen) and addresses the skill-space analogue with contribution-score retirement. Recent work (6) shows that negative constraints outperform positive directives, a reward-shaping effect (10) that Ratchet inherits by construction, since skills are synthesised from failure clusters. We evaluate on MBPP+ (11), which extends MBPP (28) with tests that discount spurious passes. The Router resembles a retrieval-and-rerank pipeline (29); our A2 ablation isolates the LLM-gate contribution from retrieval alone.

3 Method

3.1 Data model

Ratchet maintains four core artifact types (full schemas in Appendix C):

- **Skill:** a structured YAML artifact with an intent (retrieval surface), a guidance block (applies_when, key_insight, common_pitfalls, verify_before_returning), a status flag (ACTIVE/DEPRECATED/CANDIDATE), and a Cohere embed-v4 vector over its serialised form.
- **Meta-Skill:** one per evaluation suite; a Markdown document carrying a schema lock and authoring prior consumed by the Synthesizer when writing new skills.
- **Capsule:** one per (task, skill, attempt) triple; records pass/fail outcome, model output, and round index.

Table 1: Design-axis comparison across skill-library systems ($\checkmark$ = present, $\times$ = absent, $\sim$ = partial). Columns: *signal*: **SC** separate Critic, **EL** evidence log; *authoring*: **FS** failure-clustered synthesis, **TS** typed skill schema, **MS** meta-skill layer; *governance*: **PC** pattern canonicalisation, **OR** outcome-driven retirement, **AC** bounded active-cap. [†]SkillRL uses RL weight updates (different regime); included for contrast.

System	<i>signal</i>		<i>authoring</i>			<i>governance</i>		
	SC	EL	FS	TS	MS	PC	OR	AC
MemGPT (5)	$\times$	$\sim$	N/A	$\sim$	$\times$	$\times$	$\sim$	$\sim$
DSPy (14)	$\sim$	$\sim$	N/A	$\checkmark$	$\sim$	$\times$	$\times$	$\times$
Voyager (1)	$\times$	$\sim$	$\sim$	$\checkmark$	$\sim$	$\times$	$\times$	$\times$
AWM (25)	$\times$	$\sim$	$\times$	$\checkmark$	$\times$	$\times$	$\times$	$\times$
AutoManual (9)	$\times$	$\sim$	$\sim$	$\sim$	$\times$	$\times$	$\times$	N/A
ExpeL (8)	$\sim$	$\checkmark$	$\sim$	$\times$	$\times$	$\times$	$\sim$	$\times$
CASCADE (20)	$\sim$	$\sim$	$\times$	$\sim$	$\checkmark$	$\times$	$\times$	$\times$
EvolveR (18)	$\sim$	$\sim$	$\sim$	$\sim$	$\times$	$\sim$	$\sim$	$\times$
SSL (23)	$\times$	$\times$	$\times$	$\checkmark$	$\sim$	$\times$	$\times$	$\times$
AutoSkill (21)	$\times$	$\sim$	$\times$	$\checkmark$	$\times$	$\times$	$\sim$	$\times$
Strategy Genes (24)	$\times$	$\sim$	$\sim$	$\checkmark$	$\times$	$\times$	$\times$	$\sim$
SkillRL (22) [†]	$\sim$	$\sim$	$\sim$	$\sim$	$\sim$	$\times$	$\times$	$\times$
Trace2Skill (19)	$\sim$	$\checkmark$	$\sim$	$\sim$	$\times$	$\sim$	$\times$	$\times$
Ratchet (ours)	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$

- **Verdict:** one per failure capsule; carries an attribution label (HELPED/HURT/NEUTRAL/INAPPLICABLE), a short natural-language pattern string, and a confidence level.

Capsules and Verdicts form an append-only Evidence Log (SQLite). Skills are never deleted; retirement changes the status flag and removes the skill from retrieval, but its evidence remains addressable for future reinstatement.

3.2 The per-round loop

Each round proceeds through five phases (Figure 1):

1. **Eval pass:** routes only ACTIVE skills; produces the held-out capsules that the paper reports.
2. **Train pass:** runs the same pipeline on the train split (during bootstrap, CANDIDATE skills are also routed to accumulate initial evidence); failures become synthesis substrate.
3. **Critic:** for every train-failure capsule, an LLM emits a Verdict (attribution + pattern label) given the task, skill, solver output, and grader trace.
4. **Synthesizer:** reads verdicts from the last W rounds ($W=6$) and may author new skills:
 - *Pattern canonicalisation:* a union-find over pairwise cosine similarity ($\tau_{\text{canon}}=0.85$) collapses near-duplicate pattern labels before clustering.
 - *Evidence clustering:* verdicts are grouped by canonical pattern; clusters with ≥ 3 members become synthesis targets.
 - *Already-covered guard:* clusters whose canonical pattern has cosine ≥ 0.85 to an existing ACTIVE skill are skipped.
 - *Synthesis call:* the LLM is prompted with the active meta-skill (schema lock + authoring prior) and the cluster’s capsules; output YAML is validated, bank-deduplicated, length-capped, and promoted to ACTIVE.
5. **Curator:** computes per-skill contribution $\hat{c}(s) := (\text{successes} - \text{failures})/\text{trials}$ from the evidence log. A skill is retired once $n(s) \geq N_{\text{min}}$ trials have accumulated *and* $\hat{c}(s) \leq -\tau$. A hard active-cap C forces eviction of the lowest-contribution skill when synthesis would exceed the cap.

A **Meta-Synth** phase runs on a separate, slower cadence to rewrite the meta-skill from recent verdicts; in the Default it is disabled so the initial meta-skill persists unchanged. All roles are played by the same frozen LLM via role-specific prompts; durable state resides only in SQLite.

3.3 The Router

Given a task and the ACTIVE skill set, the Router selects one skill or NONE. When the bank is small ($|\text{ACTIVE}| \leq 20$) the LLM sees every skill and makes a binary choice. When the bank is larger, a two-stage retrieval (tf-idf top- $K \cup$ embedding top- K , $K = 10$) produces a shortlist that the LLM then adjudicates.

3.4 Rollback with a persistence gate

A round whose held-out pass@1 falls by more than $\tau_{\text{rb}} = 0.10$ below the running best is flagged as a regression. Rollback fires only after five *consecutive* regressions, preventing a single noisy round from triggering a restore. On rollback, the skill-bank snapshot from the best round is reinstated and the regressed state is archived for audit.

3.5 Non-divergence under bounded cap and threshold retirement

Prior systems (1; 8; 9) provide no guarantee that performance on a fixed task distribution cannot degrade over time. Ratchet’s bounded active-cap C and retirement threshold τ yield a simple non-divergence property: the argument establishes a floor (not a convergence rate) but rules out unbounded library growth as a failure mode.

Let $\mathcal{S}_t \subseteq \mathcal{S}$ denote the ACTIVE skill set at the start of round t , with $|\mathcal{S}_t| \leq C$ by construction. For a task x drawn from a fixed distribution $\mathcal{D}$, let $p_0(x) \in [0, 1]$ be the pass probability of the base LLM with no skill injected (the NONE route) and let $p(x | s)$ be the pass probability when skill s is injected. Define the per-skill contribution $c(s) := \mathbb{E}_{x \sim \mathcal{D}}[p(x | s) - p_0(x)]$. Our Curator retires skill s once $n(s) \geq N_{\min}$ trials have been accumulated and $\hat{c}(s) \leq -\tau$, where $\hat{c}(s)$ is the empirical estimator.

Proposition 1 (Non-divergence in expectation). Assume (i) the Router is retrieval-consistent: conditional on a task x , it selects either NONE or some $s \in \mathcal{S}_t$, and injecting NONE produces $p_0(x)$; (ii) $\hat{c}$ is an unbiased consistent estimator of c ; (iii) $N_{\min}$ is chosen so that with probability at least $1 - \delta$, $|\hat{c}(s) - c(s)| \leq \epsilon$ for every ACTIVE skill after $N_{\min}$ trials (a Hoeffding bound). Then the expected eval pass@1 under Ratchet is lower-bounded by $\mathbb{E}[p_0] - (\tau + \epsilon) - C\delta$. In words: the population-level pass rate cannot drift below the no-skills floor by more than a fixed margin that depends only on the retirement threshold, the estimation tolerance, and the active-cap.

Proof sketch. With probability at least $1 - C\delta$ (union bound over $|\mathcal{S}_t| \leq C$ skills), every surviving skill has $c(s) \geq \hat{c}(s) - \epsilon \geq -\tau - \epsilon$. On the high-probability event, any task routed to a surviving skill has expected pass probability at least $p_0(x) - \tau - \epsilon$; tasks routed to NONE have expected pass probability $p_0(x)$. Taking expectations over $\mathcal{D}$ gives the stated bound. $\square$

Numerical instantiation. With our Default values $\tau = 0.10$, $N_{\min} = 100$, $C = 50$, and $\delta = 10^{-3}$ per skill, the Hoeffding bound gives $\epsilon = \sqrt{\ln(2/\delta)/(2N_{\min})} \approx 0.20$ and $C\delta = 0.05$. The resulting floor is $\mathbb{E}[p_0] - 0.35$, finite only because C and τ are finite. The margin is loose at our scale (the empirical system never degrades), but on larger banks or noisier tasks $\mathbb{E}[p_0]$ may be higher and the bound becomes binding.

Remarks. (1) This is a non-divergence property, not a convergence rate: it bounds the extent of library drift, not whether the system will improve. The empirical +0.328 gain (Sec. 4.2) shows the bound is loose in practice. (2) Systems with unbounded C and no τ (1; 8; 9) have no finite analogue: the bound collapses.

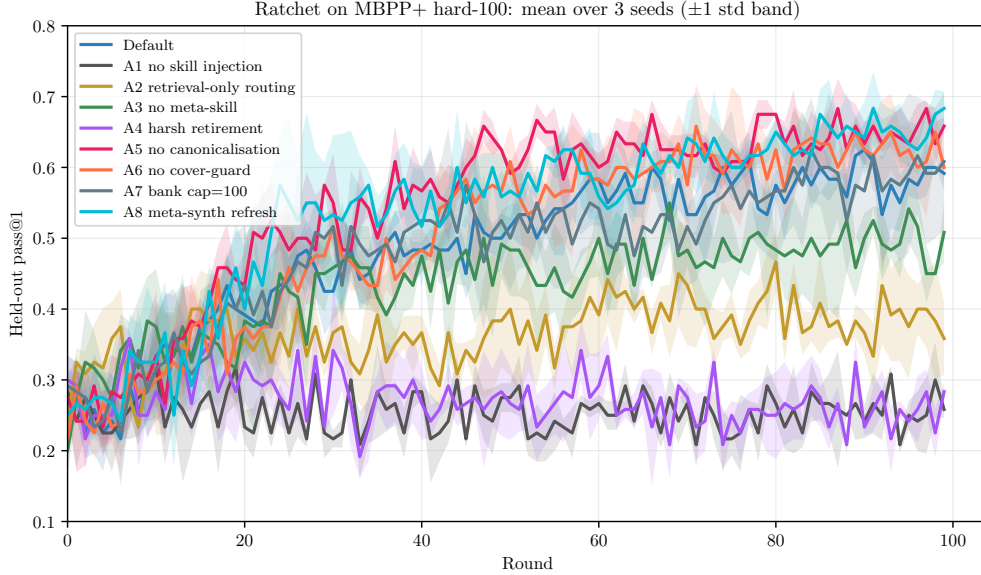


Figure 2: Held-out pass@1 by round on MBPP+ hard-100, averaged over 3 seeds (± 1 std bands). A1 (no skill injection) is the flat floor; A2 (retrieval-only) and A3 (no meta-skill) show partial gains; A4 (harsh retirement) is harmful. A5–A6 (relaxed dedup) and A8 (meta-synth refresh) slightly exceed the Default; A7 (doubled cap) is comparable with higher variance. A8’s gain comes at 55% more wall time (Table 6).

4 Experiments

4.1 Protocol

We evaluate on **MBPP+ hard-100**, a 100-task subset of MBPP+ (60 train / 40 held-out eval) selected for genuine difficulty under Claude Opus 4.7 (construction in Appendix B). The split is fixed across all conditions so round-0 scores are directly comparable. All LLM calls use **Claude Opus 4.7**; all embeddings use **Cohere embed-v4** (both via Amazon Bedrock). The Solver is a single direct LLM call with no execution feedback, no self-refinement, and no tool use, isolating the skill library’s contribution from inference-time scaffolding. Each run is 100 rounds; we report mean $\pm$ std across 3 seeds ($s=42, 7, 13$). Full hyperparameters are in Appendix A.

Metric. We report the *rolling gain*: $\text{mean}(\text{last 10 rounds}) - \text{mean}(\text{first 10 rounds})$ of held-out pass@1, plus the peak over all 100 rounds.

4.2 Main result

Table 2 and Fig. 2 summarise the results. The *Default* configuration represents our principled design choices: outcome-driven retirement with a conservative evidence floor, embedding-based deduplication, a meta-skill authoring prior, and a bounded active cap, motivated by the non-divergence analysis (Prop. 1) and standard LLM-generation hygiene. It converges above 0.58 (mean peak 0.658 ± 0.042); the no-skill control (A1) drifts at 0.25–0.30 across all seeds. Retrieval-only (A2) captures roughly a quarter of the lift, confirming that the Router’s LLM gate contributes meaningfully beyond tf-idfUembed retrieval.

4.3 Ablations: validating the default configuration

The Default configuration is designed from first principles: retire skills only after sufficient evidence (Prop. 1), deduplicate via canonicalisation, constrain authoring via a meta-skill, and bound the active bank. Eight adjustments (A1–A8) each modify one knob to test whether the corresponding design choice is load-bearing (3 seeds per condition; full settings in Table 3). We group them into two

Table 2: MBPP+ hard-100 results (mean $\pm$ std over 3 seeds). Gain = mean(last 10 rounds) – mean(first 10) of held-out pass@1; Peak = max over 100 rounds. Round-0 baselines vary across conditions because no skill is active at round 0 in any condition and each eval is a single $T=1$ sample per task: with $n=40$ tasks and $p \approx 0.25$, the per-run standard deviation is $\sqrt{p(1-p)/n} \approx 0.07$, so the observed ± 0.05 spread is within expected sampling noise. The rolling gain is computed *within* each run, cancelling this baseline variation. A1–A8 each adjust one knob from the Default.

Condition	Baseline (round 0)	Peak	Rolling gain
Default	0.258 ± 0.047	0.658 ± 0.042	$+0.328 \pm 0.018$
A1 no skill injection	0.283 ± 0.031	0.375 ± 0.000	$+0.002 \pm 0.005$
A2 retrieval-only routing	0.242 ± 0.012	0.492 ± 0.042	$+0.077 \pm 0.065$
A3 no meta-skill	0.200 ± 0.035	0.592 ± 0.047	$+0.187 \pm 0.036$
A4 harsh retirement	0.300 ± 0.035	0.433 ± 0.042	-0.019 ± 0.010
A5 no canonicalisation	0.275 ± 0.020	0.708 ± 0.012	$+0.374 \pm 0.023$
A6 no cover-guard	0.217 ± 0.024	0.700 ± 0.035	$+0.363 \pm 0.033$
A7 bank cap=100	0.292 ± 0.042	0.650 ± 0.089	$+0.317 \pm 0.110$
A8 meta-synth refresh	0.250 ± 0.035	0.725 ± 0.020	$+0.372 \pm 0.017$

questions: *which components are necessary?* (A1–A3) and *how sensitive are the governance knobs?* (A4–A8).

A1–A3: which components are necessary? **A1** (no skill injection) forces the Router to NONE, establishing the floor. **A2** (retrieval-only) bypasses the LLM gate and injects the top-ranked retrieval hit. **A3** (no meta-skill) removes the meta-skill from the Synthesizer prompt. Each ablation independently measures the cost of removing one component from the Default. Removing all skill injection (A1) eliminates nearly the entire gain ($+0.002$ vs. $+0.328$). Removing the LLM gate (A2, retrieval-only) retains only $+0.077$ (24% of the Default gain). Removing the meta-skill (A3) retains $+0.187$ (57%), making it the single most valuable component, with its absence costing -0.141 (Sec. 5).

A4–A8: how sensitive are the governance knobs? **A4** (harsh retirement) lowers $N_{\min}$ to 20 and tightens τ to 0.0. **A5** (no canonicalisation) raises τ_{canon} to 1.0. **A6** (no cover-guard) disables duplicate-cluster skipping. **A7** (bank cap=100) doubles the active-cap. **A8** (meta-synth refresh) regenerates the meta-skill every 10 rounds.

A4: harsh retirement is harmful. A4 posts -0.019 ± 0.010 , worse than the no-skill floor. At $N_{\min} = 20$ and $\delta = 10^{-3}$, the Hoeffding deviation is $\epsilon = \sqrt{\ln(2/\delta)/(2 \cdot 20)} \approx 0.44$; with $\tau = 0.0$, any skill whose true contribution $c \in [-0.44, 0]$ can be retired on unlucky early draws. In contrast, the Default ($N_{\min} = 100$, $\tau = 0.10$) has $\epsilon \approx 0.20$, so skills with $c \geq -0.10$ survive in expectation. This validates the evidence-floor argument in Prop. 1: the retirement threshold is insufficient without a sufficiently large evidence floor.

A5–A7: dedup is not necessary at this scale. A5 ($+0.374 \pm 0.023$) and A6 ($+0.363 \pm 0.033$) both exceed the Default ($+0.328 \pm 0.018$) by ~ 0.04 , within $\pm 2\sigma$ overlap at $n=3$ seeds. We interpret this conservatively: canonicalisation and the cover-guard are *not necessary* given the meta-skill’s authoring guidance on a 100-task suite, rather than claiming that removing them strictly improves performance. The meta-skill enforces enough stylistic consistency that the explicit filter’s false positives discard useful skills. A7 ($+0.317 \pm 0.110$) shows comparable mean gain but substantially higher variance; with 3 seeds we cannot distinguish whether the cap is load-bearing in mean or primarily controls variance, but the evidence is consistent with a non-binding cap at this bank size.

A8: meta-synth refresh adds cost for marginal gain. A8 refreshes the meta-skill every 10 rounds (vs. the Default’s fixed meta-skill). It achieves $+0.372 \pm 0.017$ rolling gain and the highest peak (0.725), slightly exceeding the Default within noise. However, it synthesises 24% more skills (188 vs. 152) and takes 55% longer wall time (10.1 h vs. 6.5 h), with LLM calls comparable ($+125$, $<1\%$). The extra synthesis rounds are dominated by the curate phase (27% vs. 23% of wall time). Conclusion: more frequent refresh does not meaningfully improve the learning curve but incurs substantial compute overhead.

4.4 Skill library dynamics

Over 100 rounds the Default synthesises ~ 152 candidate skills; 89 are deprecated by the curator and 50 remain ACTIVE at the cap. The Router engages a skill on $\sim 73\%$ of eval tasks, confirming that the bank is used rather than merely accumulated. Full per-condition operational metrics (wall time, LLM calls, router engagement, bank turnover) are in Appendix F.

4.5 Generality: agentic solvers on SWE-bench

To test whether Ratchet transfers beyond single-call code generation, we run the Default configuration’s full per-round loop (router, solver, critic, synthesizer, curator) on a 150-task hard subset of SWE-bench Verified (30), selected analogously to the MBPP+ subset (tasks not solved on every baseline seed, stratified by difficulty and balanced across 10 repositories; details in Appendix B), replacing the single-call solver with an agentic Claude Code³ session that can browse files, run tests, and iterate. Skills are injected as a CLAUDE.md preamble read by the agent at startup. Across three seeds, the no-skill baseline (round 0) achieves a mean held-out pass@1 of 0.65. The hard-150 filter retains tasks the no-skill agent fails on at least one of five probe seeds (independent of the three experiment seeds), not tasks it fails consistently. With Claude Code as a strong baseline agent, many retained tasks have non-zero solve rates; the 0.65 reflects the seed-to-seed variance the filter targets (tasks the agent solves some of the time but not reliably), rather than absence of difficulty. This is precisely the failure mode skills should address: injecting consistent guidance to lift solve rate from “sometimes” to “reliably.” After skill injection the mean peaks at 0.87 (best seed 0.92) within the first 20 rounds (per-seed breakdown in Table 5), confirming a +0.22 peak lift on the held-out eval split. For comparison, the MBPP+ Default achieves a +0.40 peak gain (0.258 $\rightarrow$ 0.658); SWE-bench is reported as peak only because 20 rounds is too short to compute a stable late-window mean. Both results validate that Ratchet’s hygiene recipe generalises to multi-step, tool-using agents without architecture changes. We run 20 rounds (vs. 100 on MBPP+) due to per-round cost: each agentic session averages ~ 5 min per task, so one round over 150 tasks takes ~ 50 min wall-clock at parallelism 16.

5 Discussion

The meta-skill as implicit deduplication. Removing the meta-skill (A3) costs 43% of the Default’s gain, while removing the explicit canonicalisation filter (A5) or cover-guard (A6) does not hurt, and may slightly help. The explanation is consistent: the meta-skill’s authoring prior enforces stylistic homogeneity across generated skills, making cosine-based deduplication reliable without an explicit filter. When both are present, the filter’s false positives can discard useful skills. Canonicalisation and the meta-skill are therefore substitutes, not complements at this scale, a design principle relevant to any system with LLM-authored artifacts and embedding-based retrieval.

Expected vs. unexpected results. Of the eight ablations, three produced surprises. A4 fell below the no-skill baseline. We expected harsh retirement to underperform, but not to actively harm, confirming that the evidence floor is load-bearing even in principle. A5/A6 slightly exceeded the Default, refuting our design assumption that explicit dedup would help: the meta-skill already enforces enough homogeneity. A8 yielded only marginal gain despite 55% more wall time, challenging the intuition that a stale meta-skill limits synthesis quality. The remaining five ablations behaved as designed: A1–A3 confirmed that each pipeline stage contributes, and A7 showed the cap is non-binding at this bank size.

Evidence over self-report. Skills are born from externally-diagnosed failure clusters, not from the model’s self-assessment that a skill would help. Separating diagnosis (Critic) from generation (Synthesizer) is what makes the evidence log trustworthy and retirement auditable.

Scale-dependence of hygiene knobs. The Default configuration is broadly beneficial: every condition except A4 (harsh retirement) substantially outperforms the no-skill baseline, and the non-divergence guarantee (Prop. 1) holds regardless of task distribution. However, the *exact* knob settings are scale-dependent. On our 100-task suite, the meta-skill alone provides enough stylistic consistency

³<https://www.anthropic.com/product/claude-code>

that explicit canonicalisation (A5) and cover-guard (A6) are not necessary; their false positives slightly outweigh their filtering benefit. On larger or more diverse suites, where the meta-skill cannot cover every failure mode and the bank sees hundreds of candidate patterns, we expect explicit deduplication to become load-bearing. The principled approach is to treat the Default as a safe starting point whose governance knobs (particularly τ_{canon} and the cover threshold) can be relaxed once the operator observes that the meta-skill suffices for their domain.

Limitations

Amplifier, not discoverer. Weights are frozen; Ratchet redirects existing capabilities but cannot teach genuinely new knowledge. We expect $\text{pass}@k$ to be unchanged; future work should confirm.

Scope. (i) The SWE-bench result is preliminary (20 rounds). (ii) Single model and provider; cross-model stability is not established. (iii) Critic noise can crystallise into persistent skills; mitigated via a closed label set but lacking principled calibration. (iv) Retirement measures correlation, not causation.

Ethics and safety. Closed benchmark, deterministic grading, no user data. The only artifacts are natural-language skills.

6 Conclusion

The bottleneck in self-evolving LLM skill libraries is not the author; it is the librarian. Ratchet demonstrates this by holding the author fixed (a frozen LLM, no weight update) and varying only the lifecycle management: outcome-driven retirement, a bounded active-cap, and an authoring prior that implicitly deduplicates. This minimal prescription more than doubles held-out $\text{pass}@1$ on genuinely difficult tasks (+0.328 on MBPP+), whereas the same author with no librarian exhibits library drift (+0.002). The same recipe transfers unchanged to an agentic solver on SWE-bench Verified (+0.22). Eight ablations decompose what “librarian” means: retirement and the authoring prior are load-bearing; explicit deduplication is subsumed at this scale; and the non-divergence guarantee is scale-free (finite whenever cap and threshold are finite). All hyperparameters, prompts, and subset construction details are reported for end-to-end reproducibility.

References

- [1] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [2] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026.
- [3] Xing Zhang, Guanghui Wang, Yanwei Cui, Wei Qiu, Ziyuan Li, Bing Zhu, and Peiyang He. Experience compression spectrum: Unifying memory, skills, and rules in LLM agents. *arXiv preprint arXiv:2604.15877*, 2026.
- [4] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.
- [5] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [6] Xing Zhang, Guanghui Wang, Yanwei Cui, Wei Qiu, Ziyuan Li, Bing Zhu, and Peiyang He. Do agent rules shape or distort? guardrails beat guidance in coding agents. *arXiv preprint arXiv:2604.11088*, 2026.
- [7] Richard S. Sutton. The bitter lesson. <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>, 2019. Blog post, March 13, 2019.
- [8] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

- [9] Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. AutoManual: Generating instruction manuals by LLM agents via interactive environmental learning. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [10] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. *International Conference on Machine Learning*, 1999.
- [11] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems*, 36, 2023.
- [12] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2023.
- [13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [14] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- [15] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, 2024.
- [16] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic “differentiation” via text. *arXiv preprint arXiv:2406.07496*, 2024.
- [17] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.
- [18] Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, et al. Self-evolving LLM agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*, 2025.
- [19] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Xiaoxi Jiang, and Guanjun Jiang. Trace2Skill: Parallel inductive skill distillation for LLM agents. *arXiv preprint arXiv:2603.25158*, 2026.
- [20] Xu Huang, Junwu Chen, Yuxing Fei, Zhuohan Li, Philippe Schwaller, and Gerbrand Ceder. CASCADE: Cumulative agentic skill creation through autonomous development and evolution. *arXiv preprint arXiv:2512.23880*, 2025.
- [21] Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, et al. AutoSkill: Experience-driven lifelong learning via skill self-evolution. *arXiv preprint arXiv:2603.01145*, 2026.
- [22] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. SkillRL: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.
- [23] Qiliang Liang, Hansi Wang, Zhong Liang, and Yang Liu. From skill text to skill structure: The scheduling-structural-logical representation for agent skills. *arXiv preprint arXiv:2604.24026*, 2026.
- [24] Junjie Wang, Yiming Ren, and Haoyang Zhang. From procedural skills to strategy genes: Towards experience-driven test-time evolution. *arXiv preprint arXiv:2604.15097*, 2026.
- [25] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024.
- [26] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

- [27] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- [28] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [29] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [30] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024.

A Hyperparameters and runtime configuration

Each ablation (A1–A8) overrides exactly the knob named after it and holds everything else at the values in Table 3. Full defaults will be available in the released codebase.

Table 3: Key Default-configuration hyperparameters. Omitted knobs (retry budgets, solver-cache flags, logging cadence) use the released-codebase defaults and do not affect the reported results.

Knob	Value	Purpose
<i>Skill bank</i>		
active-cap C	50	max ACTIVE skills (code default 100; Default overrides to 50; A7 uses 100)
retirement evidence floor $N_{\min}$	100	trials required before retirement can fire
retirement threshold τ	0.10	skill retired when $\hat{c}(s) \leq -\tau$
pattern-canonicalisation threshold	0.85	cosine for union-find over pattern labels
already-covered threshold	0.85	cosine vs. ACTIVE <code>applies_when</code>
bank-dedup threshold	0.85	cosine over full skill YAML
skill YAML length budget	1500 ch	resynth if output overshoots
<i>Synthesizer / Critic windows</i>		
synth lookback	6 rounds	Evidence Log window for clustering
min cluster size for synthesis	3	failures must share a canonical pattern
max skills per round	2	cap on new skills synthesised each round
<i>Router</i>		
full-bank cutoff	20	below this, LLM sees every ACTIVE skill
tf-idf / embed shortlist size K	10 each	retrieval union sent to the LLM gate
<i>Loop control</i>		
rounds	100	fixed per run
seeds	42, 7, 13	three independent seeds per condition
rollback threshold τ_{rb}	0.10	eval regression depth that counts as a regression
rollback persistence	5 rounds	consecutive regressions before snapshot restore

Temperature. All LLM calls use $T = 1.0$ (API-enforced for Claude Opus 4.7).

Compute. Each run is 100 rounds on the hard-100 subset; wall-time per run is 2–10 h depending on condition (Table 6). A1 (no-skills) is cheapest (2.3 h) because skills never inject. A8 (meta-synth-refresh) is most expensive (10.1 h) because the frequent meta-skill refresh triggers additional synthesis rounds. The Default runs in 6.5 h ($\sim 14.5\text{k}$ LLM calls, of which 69% are the solver). Our ablation sweep (A1–A8) $\times 3$ seeds launches three child processes per ablation in parallel; total wall-time for the sweep is dominated by the slowest seed within each group.

B Subset construction

MBPP+ hard-100. MBPP+ (11) extends the 378-task MBPP+ test split with additional unit tests that discount spurious passes. We construct hard-100 by the following procedure:

1. For every task in the 378-task split, run Claude Opus 4.7 under a no-skill baseline across 5 probe seeds (independent of the 3 experiment seeds), recording each seed’s pass/fail.
2. Discard tasks the baseline solves on all 5 seeds (~ 273 tasks), since a skill library cannot possibly improve on an already-saturated task. Retain the remainder (those that fail at least once); these are the tasks where a skill library could plausibly help.
3. Randomly sample 100 tasks from the retained pool (fixed random seed for reproducibility); split 60/40 into train and eval subsets.

The resulting subset is 100 tasks (60 train, 40 eval) and is consumed verbatim by every run in this paper. Reporting on a fixed hard subset lets round-0 capsules be directly comparable across conditions; reporting on the full MBPP+ split would inflate the baseline (most tasks are already solved by the base LLM) and obscure the per-round learning signal.

SWE-bench Verified hard-150. SWE-bench Verified (30) contains 500 human-validated issue-patch pairs. We construct hard-150 analogously:

1. Run the Claude Code agent (no skills) on all 500 tasks across 5 probe seeds (independent of the 3 experiment seeds); discard tasks solved on every seed.
2. From the retained pool, sample 150 tasks stratified by repository (10 repos) and difficulty, using a fixed random seed; split 90/60 into train and eval subsets.

C Skill and meta-skill schemas

A skill is a dataclass with the following fields:

```
id:          str          # snake_case, LLM-proposed
name:        str          # short human label
version:     str          # incremented on resynth
intent:      str          # canonical query-match surface
description: str          # one-line role of the skill
signals_match: list[str]  # retrieval-time lexical cues
preconditions: list[str]  # applicability conditions
tags:        list[str]    # domain tags
guidance:    Guidance     # structured injection (below)
status:      SkillStatus  # ACTIVE | DEPRECATED | CANDIDATE
embedding:   list[float]  # Cohere embed-v4 over full YAML
created_at:  timestamp
updated_at:  timestamp
```

The Guidance sub-dataclass is what the Solver actually sees when the skill is selected:

```
applies_when:      str          # when the skill applies
key_insight:       str          # what to keep in mind
common_pitfalls:   list[str]    # failure modes to avoid
verify_before_returning: str    # post-condition check
```

The `common_pitfalls` list is what concretely realises the *pitfall-oriented* framing introduced in Sec. 1: because each skill is synthesised from a cluster of failure patterns labelled by the Critic, the pitfalls are populated with the specific error modes the cluster exhibits, and the other Guidance fields read as “to avoid pitfall P on tasks where X, do Y and verify Z” rather than as unconditional directives.

A meta-skill is a Markdown document (.md) with a small YAML frontmatter block followed by “## Do” and “## Don’t” sections, carrying a *schema lock* (the field names a valid skill must have) and an *authoring prior* (stylistic guidance for the Synthesizer). At most one meta-skill per suite is ACTIVE at any time; creating a new one retires the prior. The *default* MBPP+ meta-skill, shipped ACTIVE at round 0 of every run (and left untouched through all 100 rounds in the Default; A8 refreshes it every 10 rounds). The template structure is:

```
---
id: default_mbpp
description: Authoring guidance for MBPP+ pattern-level skills.
suite: mbpp
status: active
---
## Scope
<one sentence describing the task domain>

## Do
- <field_name>: <what to write and a concrete example>
  ... (one bullet per Guidance field)
```

```
## Don't
- <anti-pattern to avoid>
... (3-5 bullets)
```

For example, the MBPP+ meta-skill’s ## Do section specifies that `signals_match` should contain 2–4 pattern words (e.g. `sliding_window`, `two_pointer`), that `common_pitfalls` must cite observed failure modes from the trace, and that `verify_before_returning` must name a concrete post-check. The ## Don’t section prohibits vague guidance, third-party libraries, and multi-pattern skills.

The A3 ablation (no-meta) removes this document from the Synthesizer’s prompt. Its effect on the learning curve (−0.141 rolling-mean gain relative to the Default) is the direct measurement of what the authoring prior contributes.

D Prompt templates

We reproduce the load-bearing prompt templates below. Full versions (including retry-repair variants) will be included in the released codebase.

Critic verdict prompt. Given a failed Capsule (task, skill if any, solver output, grader output), the Critic is asked to emit one Verdict:

SYSTEM: You are a Critic. Given a task, the skill the Router chose (if any), the solver’s code, and the grader’s verdict, judge whether the skill helped, hurt, was neutral, or was inapplicable, and label the failure by pattern.

OUTPUT: YAML with fields:

```
attribution: HELPED | HURT | NEUTRAL | INAPPLICABLE
pattern:      short string - what kind of failure is this?
confidence:   LOW | MEDIUM | HIGH
reason:       one sentence justifying the attribution
```

Synthesizer skill prompt. Given a cluster of failure Capsules sharing a canonical pattern, the Synthesizer authors one skill YAML:

SYSTEM: You author one skill YAML that would help the Solver avoid the failure pattern in this cluster. The authoring guidance below is the current meta-skill.

AUTHORING GUIDANCE: {authoring_guidance}

CLUSTER: {capsule_summary}

ACTIVE BANK (do not duplicate): {active_bank}

OUTPUT: YAML skill under {char_budget} characters. The ‘guidance’ field is a mapping of `applies_when`, `key_insight`, `common_pitfalls`, `verify_before_returning`; populate at least `applies_when` plus one of `key_insight` / `common_pitfalls`.

In the A3 ablation (−no-meta-injection), `authoring_guidance` is forced to the empty string.

E Per-seed results

Table 2 in the main text reports mean±std over 3 seeds for MBPP+ hard-100. Table 4 here gives the per-seed breakdown for all conditions.

Table 5 gives the per-seed breakdown for the SWE-bench Verified experiment (Section 4.5).

Table 4: Per-seed rolling-mean gain and peak on MBPP+ hard-100 (eval split, 40 tasks).

Condition	Seed	Baseline	Peak	Rolling gain
Default	42	0.225	0.675	+0.303
	7	0.225	0.600	+0.340
	13	0.325	0.700	+0.343
A1 no skill injection	42	0.325	0.375	+0.003
	7	0.275	0.375	−0.005
	13	0.250	0.375	+0.008
A2 retrieval-only routing	42	0.250	0.475	+0.148
	7	0.250	0.550	+0.095
	13	0.225	0.450	−0.010
A3 no meta-skill	42	0.175	0.525	+0.148
	7	0.250	0.625	+0.177
	13	0.175	0.625	+0.235
A4 harsh retirement	42	0.325	0.450	−0.005
	7	0.325	0.375	−0.027
	13	0.250	0.475	−0.025
A5 no canonicalisation	42	0.300	0.725	+0.342
	7	0.275	0.700	+0.385
	13	0.250	0.700	+0.395
A6 no cover-guard	42	0.200	0.725	+0.365
	7	0.200	0.725	+0.402
	13	0.250	0.650	+0.322
A7 bank cap=100	42	0.350	0.525	+0.172
	7	0.250	0.700	+0.340
	13	0.275	0.725	+0.440
A8 meta-synth refresh	42	0.200	0.750	+0.365
	7	0.275	0.700	+0.355
	13	0.275	0.725	+0.395

Table 5: Per-seed results on SWE-bench Verified hard-150 (eval split, 60 tasks).

Seed	Baseline	Peak	Peak lift
7	0.67	0.92	+0.25
13	0.68	0.85	+0.17
42	0.60	0.85	+0.25
Mean	0.65	0.87	+0.22

F Operational cost and skill dynamics (MBPP+)

Table 6 reports wall-clock time, LLM call counts, router engagement, and skill-bank turnover for each MBPP+ condition, averaged over 3 seeds. All runs use 100 rounds on the hard-100 subset (60 train + 40 eval tasks per round). Wall-clock times depend on API throughput and concurrency at measurement time; LLM call counts are the provider-independent cost metric.

Key observations. (1) The solver dominates LLM calls in all conditions (63–99%); the skill-management overhead (critic + synth) is at most 37% of total calls. (2) A1 still synthesises 94 skills because the full pipeline (critic, synthesizer, curator) runs normally; only the Router is forced to NONE, so skills are never injected into the solver. This isolates the routing effect: A1’s flat learning curve shows that skill creation alone, without injection, produces no gain. The 0 critic calls confirm no train capsule triggers a verdict (since no skill is ever routed). (3) A2 has the highest LLM cost (15 910) because retrieval-only routing always injects a skill, triggering a critic verdict on every train task. (4) A4 reaches only 2 active skills: harsh retirement empties the bank, explaining its negative

Table 6: Operational metrics per condition (mean over 3 seeds). *LLM total* = solver (10 000, constant) + critic (1 per routed train capsule) + synth (1 per skill born). *Rtr %* (Router %) = fraction of eval tasks where a skill was injected. *Born* = skills synthesised by the LLM; *Retired/Active* = bank state at round 100. $Born > Active + Retired$ when the curator’s dedup or cover-guard filters reject candidates before bank entry (e.g. A5 disables canonicalisation so nothing is filtered: $Born = Active + Retired$).

Condition	Wall (h)	LLM total	Critic	Synth	Rtr %	Born	Retired	Active
Default	6.5	14 451	4 299	152	73.4	152	89	50
A1 no injection	2.3	10 094	0	94	0.0	94	15	42
A2 retrieval	6.4	15 910	5 740	170	98.0	170	69	42
A3 no meta	6.3	14 843	4 676	167	79.7	167	84	50
A4 harsh retire	3.1	11 189	1 090	99	18.9	99	51	2
A5 no canon	6.5	14 520	4 393	126	80.4	126	76	50
A6 no guard	6.0	14 024	3 871	152	69.7	152	94	50
A7 cap=100	7.1	14 764	4 609	155	75.2	155	55	100
A8 meta refresh	10.1	14 576	4 388	188	73.6	188	131	50

gain. (5) A8’s 55% wall-time increase (10.1 h vs. 6.5 h) comes from extra synthesis rounds, not more LLM calls (+125, <1%).

G Code and artifact release

All experiments are fully reproducible: the paper reports every hyperparameter (Appendix A), subset construction procedure (Appendix B), and prompt template (Appendix D) needed to replicate our results from scratch using the publicly available MBPP+ and SWE-bench Verified datasets. The learning-curve figure (Figure 2) can be regenerated from the raw per-round data. We intend to release the codebase, subset JSONs, per-run logs, and figure-generation scripts.